Lab 8: K-Nearest Neighbours

Pattern Recognition and Machine Learning (EE602L) | **Name:** Jayesh | **Roll No.:** IS24BM008

> All numbers, tables and plots in this report come from one top-to-bottom run of the notebook `Lab8-IS24BM008.ipynb` (seed 42 everywhere). Rules used throughout: vote ties go to the tied class with the closest member; ties between equally good K values go to the **largest** K; ties between metric/voting combinations go to inverse-distance voting, then Euclidean.

Question 1 — Wine cultivar classification with KNN from scratch

(a) Data preparation and KNN from scratch

Split. Each class was shuffled separately with `RandomState(42)` and cut 70 % / 15 % / rest (NumPy only). The checks in the notebook confirm that no wine appears in two sets and that every wine is used exactly once.

Set	Wines	Class 0	Class 1	Class 2
Train	125	41	50	34
Validation	27	9	11	7
Test	26	9	10	7
Total	178	59	71	48

Feature	Min	Max	Range
alcohol	11.03	14.83	3.8
malic_acid	0.74	5.8	5.06
ash	1.36	3.23	1.87
alcalinity_of_ash	10.6	30	19.4
magnesium	70	162	92
total_phenols	0.98	3.88	2.9
flavanoids	0.34	5.08	4.74
nonflavanoid_phenols	0.13	0.66	0.53
proanthocyanins	0.41	3.58	3.17
color_intensity	1.28	13	11.72
hue	0.48	1.71	1.23
od280/od315_of_diluted_wines	1.27	4	2.73
proline	278	1680	1402

Ranges. Proline runs from 278 to 1680 and magnesium from 70 to 162, while hue and nonflavanoid phenols stay below 2 and 0.7. All features were standardised with the **training** mean and standard deviation only, so no information from the validation or test wines is used.

Classifier. `KNNScratch` stores the training data in `fit`. In `predict` it computes all query-to-training distances in one step: Euclidean via $\|a\|^2 + \|b\|^2 - 2a{\cdot}b$ (one matrix product) and Manhattan by broadcasting to an (n_{query}, n_{train}, d) array. There is no Python loop over test points. Neighbours are found with a stable sort, so equal distances are ordered by training index. Votes are 1 each (uniform) or 1/d (inverse-distance; d floored at 10^{-12}). **Tie rule:** if classes share the top vote total, the tied class whose member is closest to the query wins.

Check against sklearn. At K = 5 our predictions are **identical** to `KNeighborsClassifier` on all 27 validation wines (0 vote ties, so the two tie rules never had to act). As an extra check, the predictions also match for all four metric/voting combinations: yes.

(b) Choice of K and the effect of scaling

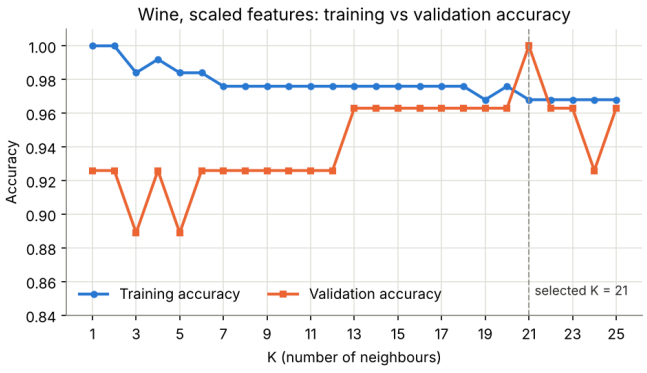


Fig. 1: Training and validation accuracy, scaled features.

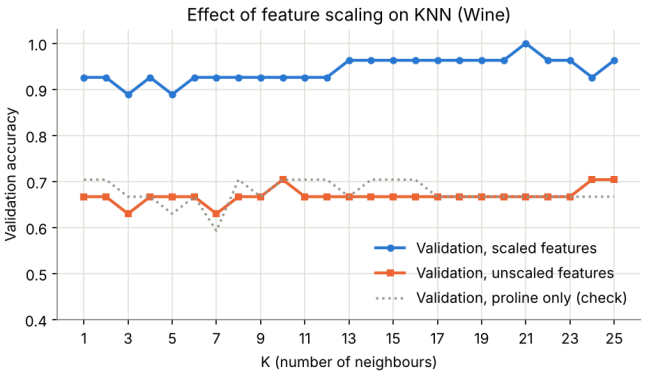


Fig. 2: Validation accuracy with and without scaling.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
Train (scaled)	1.00	1.00	.984	.992	.984	.984	.976	.976	.976	.976	.976	.976	.976	.976	.976	.976	.976	.976	.968	.976	.968	.968	.968	.968	.968
Val (scaled)	.926	.926	.889	.926	.889	.926	.926	.926	.926	.926	.926	.926	.963	.963	.963	.963	.963	.963	.963	.963	1.00	.963	.963	.926	.963
Train (unscaled)	1.00	1.00	.880	.880	.840	.840	.784	.792	.792	.768	.744	.744	.736	.768	.752	.744	.736	.752	.752	.744	.728	.744	.728	.744	.752
Val (unscaled)	.667	.667	.630	.667	.667	.667	.630	.667	.667	.704	.667	.667	.667	.667	.667	.667	.667	.667	.667	.667	.667	.667	.667	.704	.704
Val (proline only)	.704	.704	.667	.667	.630	.667	.593	.704	.667	.704	.704	.704	.667	.704	.704	.704	.667	.667	.667	.667	.667	.667	.667	.667	.667

Table: accuracy for K = 1 … 25 (Euclidean, uniform votes).

Feature	Train std	Share of sq. Euclidean distance (%)	Share of std, ~Manhattan (%)
proline	305.777	99.733	92.09
magnesium	15.149	0.245	4.56
alcalinity_of_ash	3.393	0.012	1.02
color_intensity	2.383	0.006	0.72
alcohol	0.806	0.001	0.24

Top 5 features by share of the raw distance.

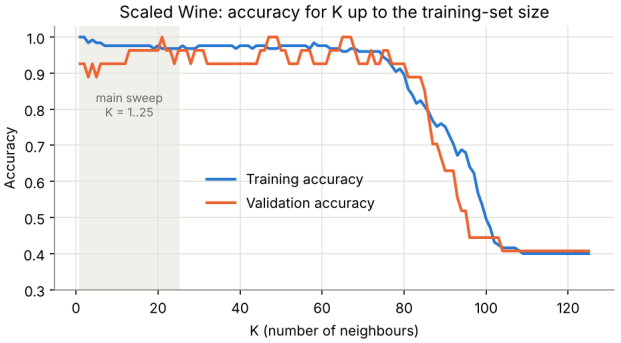


Fig. 3 (extra): scaled sweep continued to K = 125.

K	1	25	50	75	90	100	110	125
Train acc	1.000	0.968	0.976	0.944	0.752	0.496	0.400	0.400
Val acc	0.926	0.963	0.963	0.963	0.630	0.444	0.407	0.407

Choice of K. Validation accuracy is highest (**1.000**, all 27 wines correct) at **K = 21** only, so K = 21 is selected and no tie rule is needed. One honest caveat: one validation wine is worth 3.7 percentage points, and K = 13–20, 22, 23 and 25 all reach 0.963 (a single error). So K = 21 is the best by the stated rule, but any K from about 13 to 25 would be almost as good.

Why the unscaled model is worse. Without scaling, the distance is computed in raw units, so a feature with large numbers decides almost everything. Proline (training standard deviation 306) supplies **99.7 %** of the expected squared Euclidean distance between two wines; magnesium is a distant second with 0.25 %, and the other 11 features together supply less than 0.03 %. The unscaled model is therefore close to a KNN on proline alone, and the extra check confirms this: proline-only KNN has mean validation accuracy 0.676, the unscaled model 0.668, and the scaled model 0.942. Proline separates class 0 (high proline, mean about 1100) from the other two, but classes 1 and 2 overlap heavily on proline (means about 530 and 625), so the unscaled model gets stuck around 0.63–0.70 whatever K is. Scaling gives each of the 13 measurements an equal say, and the useful ones (flavanoids, colour intensity, hue, …) start to count.

Overfitting and underfitting. At K = 1 and 2 the training accuracy is 1.000 because every training wine finds itself at distance 0, while validation accuracy is only 0.926: the gap is overfitting (high variance). As K grows, the training accuracy falls to about 0.97 and validation accuracy rises to 0.96–1.00, so the gap closes. Inside K = 1 … 25 there is no clear underfitting, because the three cultivars are well separated once scaled. The extra long sweep shows it: validation accuracy stays at 0.926 or above from K = 25 to K = 75, first falls below 0.9 at K = 81, and from K = 110 the training accuracy is 0.400 and the validation accuracy 0.407. These are exactly the shares of class 1, the largest class, in each set (50 of 125 and 11 of 27), so at that point every wine gets the majority label: pure underfitting (high bias). The long sweep also reaches 1.000 again at K = 47–49 and 65–67, another sign that 27 validation wines cannot rank nearby K values reliably.

(c) Distance metric, weighting and the test evaluation

	Uniform voting	Inverse-distance voting
Euclidean	1.000	1.000
Manhattan	0.963	0.963

Validation accuracy at K = 21.

	Pred 0	Pred 1	Pred 2
True 0	9	0	0
True 1	0	10	0
True 2	0	0	7

Test confusion matrix (Euclidean, inverse-distance, K = 21).

	Precision	Recall	Support
Class 0	1.000	1.000	9
Class 1	1.000	1.000	10
Class 2	1.000	1.000	7

Per-class precision and recall (computed by hand).

Test accuracy: 1.000 (26 of 26 wines). Vote ties on validation at K = 21: Euclidean/uniform 1, Euclidean/weighted 0, Manhattan/uniform 1, Manhattan/weighted 0.

Implementation	Median predict time (ms)
Ours (NumPy, vectorised)	0.215
sklearn, algorithm=auto (kd_tree)	0.658
sklearn, algorithm=brute	0.565

Prediction time on the 26 test wines (median of 300 calls; machine-dependent).

Metric and weighting. At K = 21, Euclidean distance (1.000) beats Manhattan (0.963) by one validation wine; inverse-distance weighting gives the same accuracy as uniform voting for both metrics. Euclidean-uniform and Euclidean-weighted tie, so by the stated rule we use **Euclidean + inverse-distance weighting**. The reason is visible in the output: with uniform voting one validation wine had a vote tie and needed the tie-break rule, while weighted votes never tied.

Test result. The single test evaluation classifies **all 26 test wines correctly** (accuracy 1.000), so the confusion matrix is diagonal and precision and recall are 1.000 for every class. This should not be read as "KNN is perfect on wine": with only 26 test wines and no errors, the 95 % lower bound on the true accuracy is still only 0.868.

Speed. Both implementations do the same work (all distances from 26 test wines to 125 training wines), so the times in the table reflect overhead. In our run, our vectorised NumPy version was faster than sklearn's: on a problem this small, sklearn's fixed costs (input checks, the KD-tree search that `auto` chose) take longer than the arithmetic itself. The exact times depend on the machine and will differ slightly on Colab.

(d) Interpretation and analysis

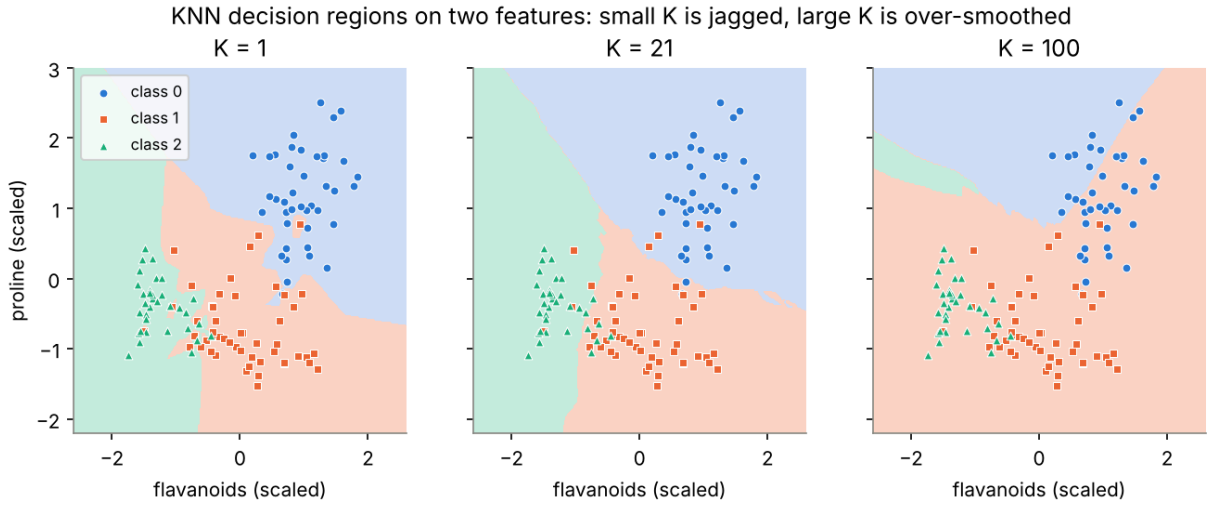


Fig. 4 (extra illustration): decision regions on two scaled features for K = 1, the selected K and K = 100.

Answer (d). With very small K (K = 1) the decision boundary follows individual training wines, so it is jagged and wraps around single unusual points: bias is low but variance is high, which is why training accuracy is 100 % while validation accuracy is lower and jumps around. As K grows, each prediction averages over more neighbours, so the boundary becomes smoother and variance falls while bias rises; at very large K the largest class takes over (in the long sweep every wine is called class 1 from K = 110), which is maximum bias. KNN has no training phase because "fitting" only stores the training data, so all the work is pushed to prediction time. To classify one wine it must compute the distance to every stored training wine and pick the closest K, so each prediction costs time proportional to (number of training points × number of features), and the whole training set must be kept in memory. One practical limitation on this data set is that KNN gives every feature the same weight in the distance, so it is only usable after scaling: on the raw measurements proline alone decides the neighbours and validation accuracy drops from about 0.94 to about 0.67.

Question 2 — KNN on synthetic data with a known ground truth

(a) KNN regression and the choice of K

300 points, $x \sim U(0, 1)$, $y = \sin(2\pi x) + \varepsilon$ with $\varepsilon \sim N(0, 0.3^2)$; random 60:20:20 split (180 / 60 / 60); sklearn `KNeighborsRegressor`.

K	1	3	5	10	20	50	100	180
train MSE	0.0000	0.0675	0.0783	0.0876	0.0954	0.1371	0.1873	0.5919
val MSE	0.1384	0.0842	0.0660	0.0590	0.0588	0.1112	0.1571	0.5187

Training and validation MSE for each K (K = 180 is the training-set size).

Selected K = 20 (lowest validation MSE). Test MSE = **0.1182**. Noise variance $\sigma^2 = 0.0900$. MSE of the true function itself: train 0.0932, validation 0.0543, test 0.1061.

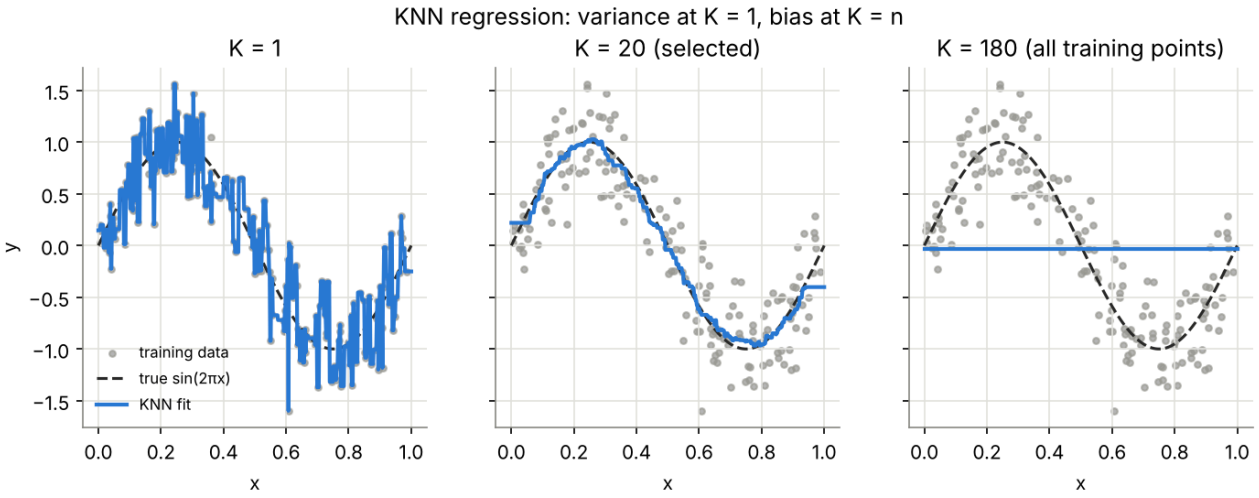


Fig. 5: fitted curves for K = 1, the selected K and K = 180, over the training data and the true function.

Choice of K. The lowest validation MSE is at **K = 20** (0.0588), only just ahead of K = 10 (0.0590); both are good choices. The test MSE at K = 20 is **0.1182**.

The two extremes.

- **K = 1** (low bias, high variance): the fit passes through every training point (training MSE 0), so it copies the noise. The curve is a jagged staircase and the validation MSE is 0.138.
- **K = 180** (all training points; high bias, low variance): every prediction is the mean of all 180 training y values, a flat line near 0. It ignores x completely, so it cannot follow the sine wave; validation MSE is 0.519, close to the variance of y itself (0.592 on the training set).
- In between, K = 10–20 averages enough neighbours to cancel much of the noise while staying local enough to follow the curve.

Best achievable test MSE vs noise. The noise has variance $\sigma^2 = 0.3^2 = 0.09$. No model can predict the noise, so on average no model can get a test MSE below 0.09. This is the irreducible error. On these 60 particular test points, even the true function sin(2πx) scores 0.106 (the test noise happened to be a little larger than 0.09), and KNN with K = 20 scores 0.118, only about 0.012 above that floor. The same effect explains why the validation MSE (0.059) is below 0.09: the 60 validation points happen to have small noise (the true function scores 0.054 on them).

(b) Distance concentration

d	Median ratio	Median d_min	Median d_max	Mean distance	√(d/6)
1	1,855.3	0.000	0.767	0.340	0.408
2	83.9	0.014	1.092	0.539	0.577
5	7.174	0.180	1.461	0.869	0.913
10	2.925	0.495	1.894	1.252	1.291
20	1.373	1.025	2.454	1.810	1.826
50	0.666	2.126	3.552	2.877	2.887
100	0.431	3.328	4.770	4.081	4.082
500	0.172	8.420	9.846	9.136	9.129
1000	0.119	12.187	13.620	12.894	12.910

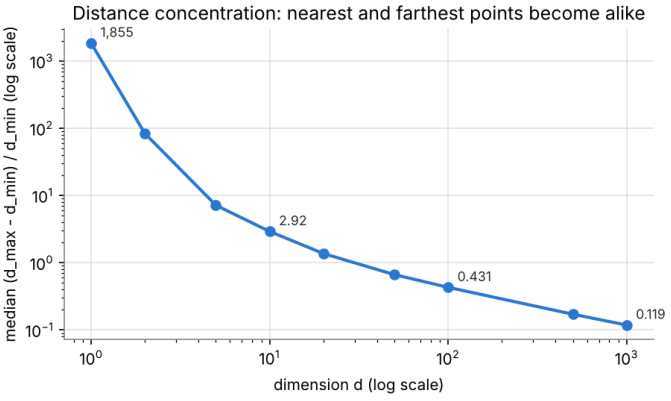


Fig. 6: median relative contrast vs dimension (log–log).

What the plot shows. The median ratio (d_max − d_min) / d_min falls from about **1855 at d = 1** to **2.9 at d = 10**, **0.43 at d = 100** and **0.12 at d = 1000**. At d = 1000 the nearest point is at distance about 12.2 and the farthest at about 13.6.

Why. A squared distance is a sum of d independent squared differences. As d grows this sum gets close to its average, so every distance is close to √(d/6) (12.9 at d = 1000, matching the table), while the spread of distances stays roughly the same size. So the relative gap between the nearest and the farthest point shrinks, roughly like 1/√d.

What it means for KNN. In high dimensions the "nearest" neighbour is only slightly closer than a random point. Small noise can change which point is nearest, and being nearest no longer means being much more similar. KNN relies entirely on this meaning, so it weakens as the dimension grows.

(c) Effect of irrelevant features

For each m, `make_classification` (2 informative + m noise columns, settings as in the sheet). Split 60:20:20 with the same stratified split as Q1 (900 / 300 / 300); standardised with training statistics after the split; K chosen in 1 ... 30 on validation (ties → largest K); test accuracy reported.

m	K (all)	KNN all	K (2 inf.)	KNN 2 informative	Logistic regression
0	30	0.983	30	0.983	0.980
2	8	0.970	7	0.983	0.973
5	30	0.963	13	0.980	0.973
10	26	0.963	19	0.983	0.973
20	20	0.930	15	0.980	0.953
50	26	0.853	23	0.983	0.973
100	26	0.787	7	0.977	0.950

Test accuracy for each m (K = value selected on validation).

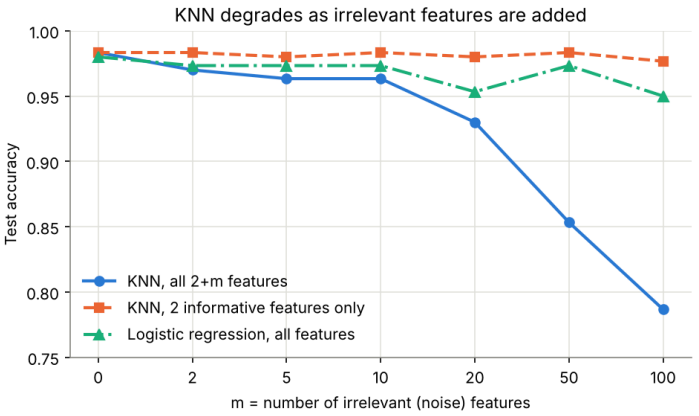


Fig. 7: test accuracy vs number of irrelevant features.

What the plot shows.

- **KNN on all features** falls steadily as noise columns are added: test accuracy **0.983 (m = 0) → 0.963 (m = 10) → 0.930 (m = 20) → 0.853 (m = 50) → 0.787 (m = 100)**.
- **KNN on the 2 informative columns only** stays between 0.977 and 0.983 for every m. The two informative columns are the same numbers for every m. The small differences come from `make_classification`'s default `flip_y = 0.01`: about 1 % of labels are assigned at random, and which ones depends on m. This label noise also explains why no model goes much above 0.98.
- **Logistic regression on all features** moves only from 0.980 to 0.950.

So the loss comes from the noise features, not from the classification problem becoming harder.

(d) Interpretation and analysis

Answer (d). Part (b) shows that as more dimensions are added, all distances become almost equal, and in (c) each noise column adds random spread to every distance that has nothing to do with the class, so the two informative columns get drowned out and the "nearest" neighbours become close to random points: KNN on all features falls from 0.983 to 0.787, while KNN on the two informative columns stays near 0.98. Logistic regression is far less affected (0.980 to 0.950) because it learns a separate weight for each feature and can push the weights of the noise columns close to zero, whereas KNN's distance gives every column the same say and has no way to learn which ones matter; changing K cannot fix this, since, as (a) showed, K only controls how much the model smooths (bias versus variance), not which features count. Two practical remedies are feature selection (keep only useful columns, e.g. chosen by mutual information or an L1-penalised model) and learning feature weights or a distance metric (e.g. Neighbourhood Components Analysis). In Question 1 the problem was different: all 13 features carried class information but on very different scales, so standardisation fixed it, whereas here every column already has the same scale and standardisation actually gives each pure-noise column as much influence as an informative one. A limitation that appears only when the training set becomes very large is prediction cost: every query must be compared with all n stored points, so prediction time and memory grow in proportion to n, and the tree indexes that speed this up lose their advantage in high dimensions.

Conclusions

- **Q1:** On the scaled Wine data, our NumPy KNN matches sklearn exactly, K = 21 with Euclidean distance and inverse-distance voting is selected on the validation set, and it classifies all 26 test wines correctly. Without scaling, proline dominates the distance and accuracy falls to about 0.67.
- **Q2:** K controls the bias–variance trade-off in KNN regression (K = 20 gives a test MSE of 0.118, close to the noise floor). In high dimensions distances concentrate, and irrelevant features steadily ruin KNN while logistic regression is barely affected.